📚 Module 35-40: Probability - Complete Notes

🎯 What You'll Learn

Master **probability** — calculating likelihood, independent/dependent events, and combined probabilities.

📖 Concept Explained

The Basics

Probability measures how likely an event is to occur.

```
Probability = Favorable outcomes / Total outcomes

P(event) ranges from 0 to 1:
    0 = Impossible
    1 = Certain
    0.5 = 50-50 chance
```

Key Concepts

```
SAMPLE SPACE: All possible outcomes
    Coin flip: {Heads, Tails}
    Die roll: {1, 2, 3, 4, 5, 6}

EVENT: Specific outcome(s) we care about
    "Rolling even": {2, 4, 6}

PROBABILITY:
    P(even) = 3/6 = 0.5 = 50%
```

Types of Events

```
INDEPENDENT: One doesn't affect the other
    Coin flip 1 doesn't affect flip 2

DEPENDENT: One affects the other
    Drawing cards without replacement

COMPLEMENTARY: P(A) + P(not A) = 1
    P(rain) = 0.3  →  P(no rain) = 0.7
```

💻 Programming Connection

Code Examples

```
# Example 1: Basic Probability
```

```python
def probability(favorable, total):
    """Calculate probability"""
    if total == 0:
        return 0
    return favorable / total

# 3 favorable outcomes out of 10
print(probability(3, 10))  # 0.3 or 30%
```

```python
# Example 2: P(A AND B) - Both happen

def prob_and_independent(p_a, p_b):
    """Probability of both A and B (independent)"""
    return p_a * p_b

# Coin lands heads AND die shows 6
p_heads = 0.5
p_six = 1/6
print(f"P(heads AND 6): {prob_and_independent(p_heads, p_six):.3f}")  # 0.083
```

```python
# Example 3: P(A OR B) - Either happens

def prob_or_exclusive(p_a, p_b):
    """Probability of A or B (mutually exclusive)"""
    return p_a + p_b

def prob_or_general(p_a, p_b, p_a_and_b):
    """Probability of A or B (general case)"""
    return p_a + p_b - p_a_and_b  # Subtract overlap!

# Die shows 1 OR 6 (mutually exclusive)
print(prob_or_exclusive(1/6, 1/6))  # 0.333

# Card is Heart OR King (not exclusive - King of Hearts!)
p_heart = 13/52
p_king = 4/52
p_king_heart = 1/52
print(prob_or_general(p_heart, p_king, p_king_heart))  # 0.308
```

```python
# Example 4: Complementary Probability

def prob_complement(p):
    """P(not A) = 1 - P(A)"""
    return 1 - p

# P(at least one success) is easier as 1 - P(all failures)
def prob_at_least_one_success(p_success, n_trials):
    p_all_fail = (1 - p_success) ** n_trials
    return 1 - p_all_fail
```

```python
# Each test has 5% chance of catching bug
# Run 10 tests, what's P(at least one catches)?
print(prob_at_least_one_success(0.05, 10))  # 0.40 (40%)
```

```python
# Example 5: Multiple Independent Events

def prob_all_succeed(probabilities):
    """P(A AND B AND C...) for independent events"""
    result = 1
    for p in probabilities:
        result *= p
    return result

def prob_any_fail(probabilities):
    """P(at least one fails)"""
    return 1 - prob_all_succeed(probabilities)

# Three tests with 95%, 90%, 85% pass rates
pass_probs = [0.95, 0.90, 0.85]
all_pass = prob_all_succeed(pass_probs)
print(f"All pass: {all_pass:.1%}")  # 72.7%
print(f"At least one fail: {1-all_pass:.1%}")  # 27.3%
```

```python
# Example 6: Conditional Probability

def prob_conditional(p_a_and_b, p_b):
    """P(A|B) = P(A and B) / P(B)"""
    if p_b == 0:
        return 0
    return p_a_and_b / p_b

# Given test failed, what's probability it's a critical bug?
# P(critical AND failed) = 0.15, P(failed) = 0.20
p_critical_given_fail = prob_conditional(0.15, 0.20)
print(f"P(critical | failed): {p_critical_given_fail:.1%}")  # 75%
```

🧪 SDET/Testing Application

```python
# SDET Scenario: Flaky Test Analysis

def analyze_flakiness(runs, failures):
    """Analyze flaky test probability"""
    p_fail = failures / runs

    # P(fails at least once in N runs)
    def p_at_least_one_failure(n):
        return 1 - (1 - p_fail) ** n
```

```
    return {
        "total_runs": runs,
        "failures": failures,
        "failure_rate": f"{p_fail:.1%}",
        "p_fail_in_5_runs": f"{p_at_least_one_failure(5):.1%}",
        "p_fail_in_10_runs": f"{p_at_least_one_failure(10):.1%}",
        "p_fail_in_20_runs": f"{p_at_least_one_failure(20):.1%}"
    }

# Test fails 2% of the time
print(analyze_flakiness(100, 2))
```

```
# SDET Scenario: Test Suite Reliability

def suite_reliability(test_reliabilities):
    """Calculate probability entire suite passes"""
    all_pass = prob_all_succeed(test_reliabilities)

    return {
        "num_tests": len(test_reliabilities),
        "individual_avg": f"{sum(test_reliabilities)/len(test_reliabilities):.1%}",
        "suite_pass_rate": f"{all_pass:.1%}",
        "suite_fail_rate": f"{1-all_pass:.1%}"
    }

# 20 tests, each 99% reliable
reliabilities = [0.99] * 20
print(suite_reliability(reliabilities))
# Suite pass rate: 81.8% (even with 99% individual!)
```

```
# SDET Scenario: Bug Detection Probability

def bug_detection_strategy(detection_rates):
    """Probability of catching bug with multiple strategies"""
    p_all_miss = 1
    for rate in detection_rates:
        p_all_miss *= (1 - rate)

    p_caught = 1 - p_all_miss

    return {
        "strategies": len(detection_rates),
        "detection_rates": detection_rates,
        "p_bug_caught": f"{p_caught:.1%}",
        "p_bug_missed": f"{p_all_miss:.1%}"
    }

# Unit tests: 60%, Integration: 40%, E2E: 30%
strategies = [0.60, 0.40, 0.30]
```

```
print(bug_detection_strategy(strategies))
# Combined detection: 83.2%
```

🎓 Practice Problems

Problem 1: Easy 🟢

Challenge: Die roll. What's P(even number)?

Problem 2: Medium 🟡

Scenario: Test has 95% pass rate. Run it 3 times.
Challenge: What's P(all 3 pass)? P(at least one fails)?

Problem 3: Application 🔴

Scenario: 100 tests, each 1% flaky. Running full suite.
Challenge: What's the probability at least one test flakes?

⚠️ Common Mistakes

❌ **Mistake 1**: Adding probabilities for AND
✅ **Fix**: AND = multiply, OR = add (then subtract overlap)

❌ **Mistake 2**: Ignoring dependence
✅ **Fix**: Check if events affect each other

🔑 Key Takeaways

✅ **P = Favorable / Total**
✅ **P(A AND B) = P(A) × P(B)** for independent
✅ **P(A OR B) = P(A) + P(B) - P(A AND B)**
✅ **P(at least 1) = 1 - P(none)**
✅ **Many 99% tests ≠ 99% suite pass rate**

💾 *Save as:* `Module_35_40_Probability.md`

Phase 7 Complete! 🚀